DEPARTMENT OF ECONOMICS AND MANAGEMENT

Degree Program

in ECONOMICS AND DIGITAL FIRM

Final Report

The Optimization of Food Delivery Systems Using Public Transport Infrastructure:

Brescia - Case study

Supervisor: Chiar.mo Prof. Valentina Morandi

Graduating student:

Selam Mahmud Ali

Serial number n. 738215

Academic Year 2023/2024

Table of Contents

1. INTRODUCTION 4

1.1 UNDERSTANDING LAST-MILE DELIVERIES 4
1.2 FOOD DELIVERY SYSTEM IN BRESCIA 5
1.3 FEATURES AND BENEFITS OF STARSHIP ROBOTS 6
1.4 INCORPORATION OF FOOD DELIVERY SERVICES IN EXISTING PUBLIC TRANSPORT INFRASTRUCTURE 7

2. ECONOMIC AND LOGISTICAL OPTIMIZATION 8

2.1 LINEAR PROGRAMMING MODEL 8
2.2 COST MINIMIZATION AND RESOURCE ALLOCATION 9
2.3 OPTIMIZATION AND THE ASSIGNMENT PROBLEM 9
2.4 COLLABORATIVE LOGISTICS AND COST SHARING 10
2.5 ENVIRONMENTAL AND SOCIAL BENEFITS 10

3. DATA 12

3.1 METRO LINE: 12
3.2. TRAM LINE: 13
3.3 TIMETABLE SCHEDULES: 14
3.3 RESTAURANTS AND CUSTOMERS 15

***4* METHODOLOGY 16**

4.3 MODEL FORMULATION 16
4.3.1 SETS AND INDICES: 16
4.3.2 PARAMETERS: 17
4.3.3 DECISION VARIABLES: 18
4.3.4 OBJECTIVE FUNCTION 18
4.3.5 CONSTRAINTS 19
4.4 PYTHON CODE AND LOGIC: 19
4.4.1 GENERATION OF THE SCHEDULE (T) 19
4.4.2 SAMPLE POPULATION 20
4.4.3 CREATION OF ORDERS 21
4.4.4 ARCS 22
4.4.5 DJIKSTRA'S ALGORITHM & THE B MATRIX 23
4.4.6 RUNNING THE MODEL 25

5. ANALYSIS 26

5.1 PEAK VS. OFF-PEAK SCHEDULES 29
5.2 CAPACITY AND FREQUENCY ANALYSIS 30
5.3 LIMITATIONS OF THE MODEL 31

CONCLUSION: 32

REFERENCES:.....33

APPENDIX 34

1. Introduction

1.1 Understanding Last-mile deliveries

The continuous movement of people and goods in the modern urban settings necessitate an efficient and connected system to support it. Technological advancements assist this initiative through platforms for online trade and commercial transactions that allow direct delivery of goods and services to clients. In recent years, it has become extremely convenient to order online rather than to take the time of day to go run errands. Online trade has simplified the daily lives of people by permitting them to order whatever they desire in the convenience of their homes. Furthermore, such online practices have expanded across various industries like retail, food, etc…and the COVID-19 pandemic has played a huge role in accelerating this trend (Asdecker, B, 2021).

In the business world, the term "last-mile delivery" is very common and it can even be considered essential. **Last-mile delivery** is the final step of the delivery system that starts from the transportation hub where goods are taken and transported to the delivery destination. Last-mile delivery is considered the most complex and costly part of the supply chain due to the need for efficiency, speed, and customer satisfaction in urban areas that are densely populated (Gaevers, R. et.al, 2011).

Companies must find an equilibrium between optimizing the routing efficiency and meeting the customer convenience. This may imply higher costs that can amount to up to 30% of the total distribution expenses (Gaevers, R. et.al, 2011). Besides this, the high frequency of freight and passenger movements in urban areas leads to increased traffic congestion, air and noise pollution, and more risk of traffic accidents (Rodrigue, J.P.,

2016). These complex challenges regarding the delivery of goods to different locations within cities require new solutions.

1.2 Food Delivery System in Brescia

This thesis explores the optimization of food delivery systems in the city of Brescia, in northern Italy, by basing the model on existing public transport routes, in particular the metro line and the new tram line T2 project (Comune Brescia). Currently, there are companies like Just Eat, Glovo, Deliveroo and Uber Eat that provide food delivery services from restaurants to clients. These companies use riders and bikers waiting near the restaurants to deliver the food. These riders need to possess bikes, e-bikes and scooters (Just Eat, n.d.) to work for the companies. They are also meant to travel on the same routes as other private vehicles. This adds to the already existing traffic congestion problem in the city.

The aim of this study is to find a solution primarily to the traffic congestion, and secondly to the efficiency of the delivery network. The suggested solution is to automate the whole process of the food delivery system, assigning orders to small autonomous robots called Starships (Starship Technologies, n.d.). These small robots will be travelling from the location of the restaurant to the nearest metro/tram, on sidewalks. This path should be within the range of traveling distance of the Starships. The Starships will then onboard the metro/tram and travel throughout the city, reaching the nearest station of the metro to the client's location. Here they will off board the metro/tram and travel to the client's location, reaching their destination.

1.3 Features and benefits of Starship robots

Starship delivery robots are autonomous, self-driving devices designed to handle last-mile deliveries of items like groceries, hot food, or parcels within a two-mile radius, which is equivalent to 3,2 kms. These robots navigate safely at a walking pace of about 4 mph, using an array of 12 cameras, sonic sensors, and computer imaging to detect and avoid obstacles. They are battery-powered and can run a full day on a single charge (Starship Technologies, n.d.). Starships represent a good example of an energy-efficient and environmentally friendly delivery option.

For doubts about efficiency regarding tampering by individuals or pedestrians on the streets, the technology of these robots features loud alarms and precise GPS tracking. They are also very heavy making them hard to pick up or steal (Starship Technologies, n.d.). Moreover, the compartment where the food is stored remains locked until the Starship reaches its destination, where the client can finally unlock it via a dedicated app, that also allows real-time tracking (Starship Technologies, n.d.).

Despite their compact size, these robots can carry up to three bags of groceries/orders, with insulation ensuring food stays at the right temperature (Starship Technologies, n.d.).

The benefits of Starship robots in food delivery networks include:

- <u>Better Efficiency in Urban Environments</u>: these automated robots reduce delays due to road congestions. This allows the customers to know exactly when the order will be arriving by providing reliable delivery estimates (Starship Technologies, n.d.).
- <u>Cost efficiency</u>: Starships also reduce labor cost associated with human delivery personnel. Objectively speaking, these robots do not need a break time, assuring the service provider continuity of the workflow. Considering both these factors,

in terms of scale economy, where the marginal cost diminishes with bigger volumes, the customer benefits of a lower final price of delivery (Starship Technologies, n.d.).

- Environmental benefits: Starship robots are electric and therefore produce no emissions during operation. This guarantees for a lower carbon footprint. Their role as an electric vehicle contributes to sustainable urban logistics (Starship Technologies, n.d.).
- Higher Delivery capacity: These robots can carry multiple orders at once (in this model up to 5), optimizing the use of transport resources (Starship Technologies, n.d.).
- Scalability and Flexibility: the number of available starships can be adjusted to the demand. During peak times, additional robots can be deployed to handle increased delivery volumes, while during quieter periods, the number of active robots can be reduced. This flexibility assures efficient resource management in varying seasons and conditions (Starship Technologies, n.d.). This factor also provides cost benefits, by avoiding expensive hiring and layoffs of employees due to seasonal demand fluctuation.

1.4 Incorporation of good delivery services in existing public transport infrastructure

The approach that is evaluated in this thesis is last-mile delivery that integrates public transportation systems into the food delivery network. This concept can be referred to as "multimodal freight" as it aligns with the increasing interests in combining passenger and cargo services to maximize efficiency. Companies can thus collaborate with transit agencies in allocating specific vehicles for parcel delivery (Holguín-Veras et al., 2018).

This approach has been implemented in several cities. A pilot project in Zurich has incorporated parcel deliveries on trams, providing practical insight into the benefits for urban logistic using public infrastructure (Kochan et al., 2020).

Similarly, some cities in China have also experimented with buses for dual purposes of public transport and goods delivery, showing early signs of success (Li et al., 2019).

2. Economic and Logistical Optimization

2.1 Linear Programming model

For an optimization of the food delivery system in Brescia we're using linear programming to optimize delivery routes and maximize the number of customers served. Below is a breakdown of how it works:

- **Defining Variables:** We must set up the variables that help understand whether a particular delivery path should be used in a metro or tram run.
- **Objective Function:** The goal is to maximize the total number of served customers.
- **Constraints:** We need to ensure that the total number of deliveries in each metro or tram run doesn't exceed the maximum capacity in each one of the arcs of the relative run. Furthermore all paths should follow the scheduled timings and constraints. The reason we consider the arcs of the runs is because the Starships may onboard in station A and offboard in station B (arc A-B), therefore freeing space in arc B-C.

The integration of public transport into food delivery systems exploits key economic principles. The central concepts used for this thesis are optimization and resource allocation. The goal is to maximize the number of deliveries while minimizing

costs such as time, labor, and vehicle use. Here, the focus is on making the best use of public transport and delivery robots (or "Starships") to boost efficiency.

2.2 Cost Minimization and Resource Allocation

From a cost-minimization perspective, we aim to cut down on the expenses typically associated with traditional delivery methods, like the labor and fuel costs for motorbike riders or cyclists. These autonomous delivery robots reduce the need for individual delivery vehicles. This approach aligns with economic theories on resource allocation (Varian, 1992). Efficiently allocating resources (in this case Starships) guarantees we can serve as many customers as possible within the constraints of transport schedules and route capacities.

In logistics, minimizing costs is very important, and the goal is to deliver to the most customers at the lowest possible cost. By applying this model in existing public infrastructure, such as metro or tram lines, we can group multiple deliveries and use the same transport route. This reduces the per-unit delivery cost, which, as mentioned above, traditionally includes labor and fuel. Instead, we exploit the underused capacity in public transport, following classic optimization models.

2.3 Optimization and the Assignment Problem

Using linear programming, we aim to maximize the number of customers served. This involves solving a constrained optimization problem where variables represent the assignment of orders to metro or tram routes. Constraints include tram or metro capacity, available delivery time, and the number of starships.

Classical economic theory defines optimization as finding the best solution within given constraints (Debreu, 1959). Here, our model seeks to maximize the satisfaction

from food deliveries, respecting the available resources and time limits. This approach aligns with Pareto efficiency, meaning that any reallocation of resources (Starships) cannot increase the number of deliveries without reducing service for another customer (Arrow, 1963).

2.4 Collaborative Logistics and Cost Sharing

The idea of cost sharing, common in collaborative logistics, supports this model. By pooling multiple deliveries onto the same metro or tram route, we can spread the fixed transportation costs over a larger number of orders. This mirrors collaborative resource-sharing models in logistics, which aim to cut costs and boost efficiency (Wang et al., 2018). Treating public transport as a shared resource helps minimize overall costs and maximize delivery efficiency.

In fact, this thesis explores the potential for using shared public transportation infrastructure between people and goods to reduce operational costs. This will, in its turn, minimize environmental impacts and enhance delivery efficiency.

Incorporating this cooperative approach within Brescia's public transport system gives way to an efficient use of resources, as food delivery robots (Starships) can travel alongside passengers, making use of the full metro and tram capacity. This not only boosts delivery efficiency but also aligns with sustainable urban logistics practices by reducing the number of motorized delivery vehicles on the road. This solution can be a promising optimization approach for urban delivery networks.

2.5 Environmental and Social Benefits

The concept of the green economy is closely linked to the optimization of food delivery systems, particularly in urban settings like Brescia, where sustainable practices

are increasingly necessary to contain environmental impacts. Green economy emphasizes reducing carbon footprints and it works toward energy efficiency, and promoting the use of renewable resources (UNEP, 2011).

In the context of this thesis, integrating public transport infrastructure with automated delivery robots represents a green economy initiative by making use of existing, low-emission public transportation systems. This approach not only reduces traffic congestion and greenhouse gas emissions but also aligns with the broader goals of sustainable urban logistics. By optimizing delivery routes using public transport, this model supports the transition to a greener urban economy, minimizing the environmental impact of last-mile deliveries (Pelletier, Jabali, & Laporte, 2016). Furthermore, the use of energy-efficient autonomous delivery robots, such as Starships, complements these efforts by providing an eco-friendly alternative to traditional delivery vehicles, contributing to the overall reduction of urban carbon emissions (Starship Technologies, n.d.).

3. Data

For this study the sample data is sourced from the infrastructure system in Brescia (Gruppo Brescia Mobilità). The data required for this study includes:

3.1 Metro line:

There are 17 metro stations in the city of Brescia, extending from the north of the city towards the east-end (Figure 1). The Metro stations and their respective coordinates are the following:

Prealpino :	*(45.58099, 10.22928),*
Casazza :	*(45.57598, 10.23030),*
Mompiano :	*(45.56894, 10.23669),*
Europa :	*(45.56413, 10.23429),*
Ospedale :	*(45.55646, 10.23349),*
Marconi :	*(45.55231, 10.22468),*
San Faustino :	*(45.54567, 10.22000),*
Vittoria :	*(45.53942, 10.21847),*
Stazione FS :	*(45.53334, 10.21455),*
Bresciadue :	*(45.52736, 10.21280),*
A2A Lamarmora :	*(45.51906, 10.22550),*
Volta :	*(45.51882, 10.22542),*
Poliambulanza :	*(45.51859, 10.23673),*
San Polo Parco :	*(45.51778, 10.24255),*
San Polo :	*(45.51966, 10.25910),*
Sanpolino :	*(45.51746, 10.26689),*
Sant Eufemia Buffalora:	*(45.51145, 10.28007).*

Table 1: Brescia Metro-line coordinates *Google maps (2024)*

3.2. Tram line:

There are limitations to the data regarding the Tram T2 line, since this line is yet to be implemented (approximately 2029 according to Comune Brescia). For this reason, most of the estimated data referring to the T2 line is used for representative purposes. Below are the stations and their coordinates, along the T2 route:

Tram 1 :	*(45.569, 10.20707),*
Tram 2 :	*(45.56498, 10.20334),*
Tram 3 :	*(45.56305, 10.1966),*
Tram 4 :	*(45.56026, 10.19167),*
Tram 5 :	*(45.55645, 10.1879),*
Tram 6 :	*(45.55224, 10.19236),*
Tram 7 :	*(45.54899, 10.19717),*
Tram 8 :	*(45.54857, 10.20412),*
Tram 9 :	*(45.54683, 10.21176),*
Tram 10 :	*(45.54472, 10.22047),*
Tram 11 :	*(45.54497, 10.22472),*
Tram 12 :	*(45.54094, 10.22326),*
Tram 13 :	*(45.53667, 10.22232),*
Tram 14 :	*(45.53228, 10.22086),*
Tram 15 :	*(45.53203, 10.21506),*
Tram 16 :	*(45.53461, 10.21136),*
Tram 17 :	*(45.53126, 10.20817),*
Tram 18 :	*(45.52559, 10.20349),*
Tram 19 :	*(45.52667, 10.19817),*
Tram 20 :	*(45.52795, 10.19355),*
Tram 21 :	*(45.52965, 10.1878),*

Tram 22 :	*(45.53088, 10.18325),*
Tram 23 :	*(45.53021, 10.17677).*

Table 2: Brescia-Tram T2 line coordinates. (approximate estimations)

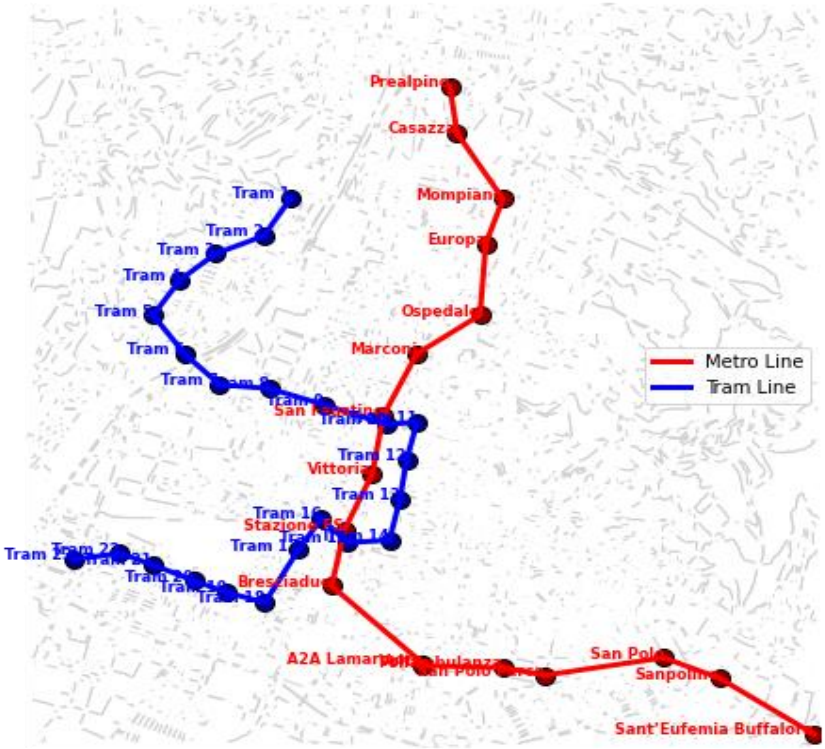

Figure 1: Metro line (red) and Tram T2 line (blue) (data {output}, APPENDIX)

3.3 Timetable Schedules:

The timetables indicating the metro line (Gruppo Brescia Mobilità) and an indicative tram timetable are used in this model to define the schedule for the runs to which the starships will be assigned. To make the model coherent, the timetable for both metro and tram are identical.

The metro runs every day from 5 AM to 12 AM (Midnight), except for Saturdays when it runs until 1 AM (Gruppo Brescia Mobilità). For simplicity, the schedule that is considered in this model runs 19 hours a day for both lines.

It has been identified that the real-time frequency of the metro/tram varies according to peak and off-peak times. The peak time is considered as being from 7 AM to 7 PM, when workers and students are supposedly commuting. Outside of this timeframe, it's considered to be an off-peak time. In peak time, the frequency of the metro and tram for the purpose of this thesis is set to 10 minutes, while on off-peak times it is approximately 20 minutes. Based on this schedule the model determines the assignment of the starships to the designated runs.

3.3 Restaurants and customers

The restaurants and customers are randomly generated (data, APPENDIX) with a normal distribution around each metro and tram station. In this model, 5 restaurants and 10 customers are generated around each station (Figure 2).

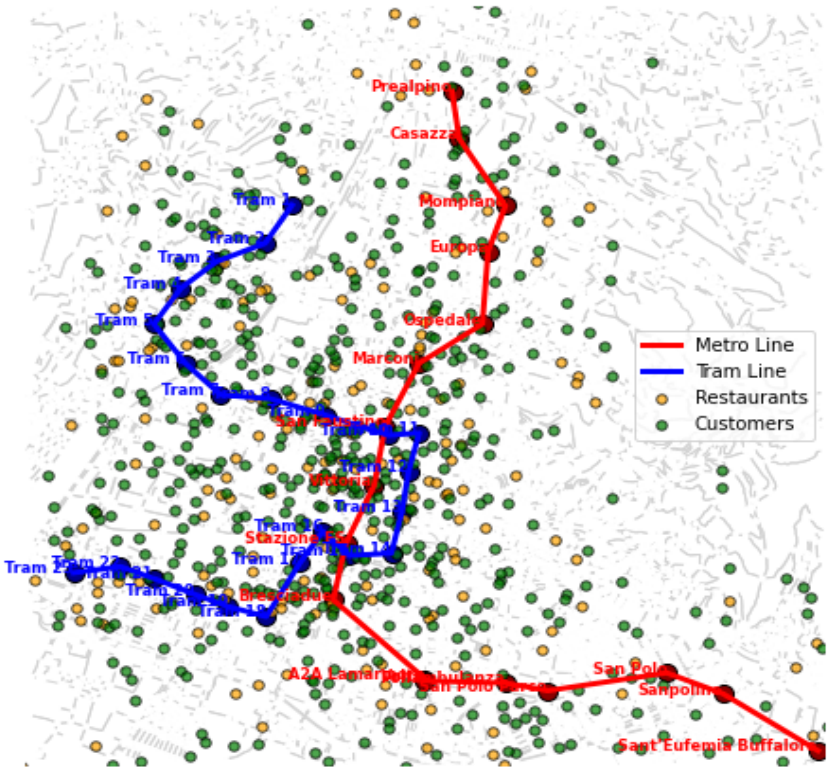

Figure 2: Customers, Restaurants and transport routes. (data {output}, APPENDIX)

4 Methodology

The purpose of the methodology section is to explain the development and execution of the model through the following steps:

- Model formulation;
- Code logic and its implementation;

4.1 Model Formulation

4.1.1 Sets and Indices:

- *R*: set of Restaurants
- *C*: set of Customers
- *T*: Timetable of metro/tram runs
- A_l: set of arcs from station n to station n+1 for each line (metro and tram)
- *Orders*: Set of possible orders {services}

The *R* and *C* sets are randomly generated with a normal distribution around the stations for the purpose of this thesis and are not realistic restaurant or customer locations.

The set *T* is the timetable schedule for the metro and tram lines. Therefore, the set varies according to the time of day. In fact, if the order is placed in peak times, the frequency of the metro and tram is higher (every 10 mins). On the other hand, in off-peak times the frequency of metro/tram runs is 20 minutes.

A_l is a set that contains all the arcs of both metro and tram lines. Each arc is defined as a pair of stations, and every arc has a fixed maximum capacity for the number of Starships.

Orders represent all possible pairs of restaurant-customer where the distance exceeds 1.6 km, as this falls within the maximum travel range of a Starship, eliminating the need for public transportation. These orders consist of services from restaurant $i \in$ R to customer $j \in C$.

4.1.2 Parameters:

- $\boldsymbol{cap_a}$: Capacity of metro/tram in arc a ($\forall$a $\in Al$). This value is set to 20.

  The capacity threshold allows to check each arc. The reason we check arcs and not just the runs is because Starships have different paths and may offboard at each station, liberating space for other Starships to onboard at subsequent stations of the same run. Even though the capacity in this model is set to a fixed value (20), it is better to keep it as a parameter relative to the model. This choice is justified by future prospect of scaling according to demand and available infrastructure.

- $\boldsymbol{d_o}$: Answers how many starships are required for the service o $\in Orders$ (from restaurant i $\in R$ to customer j $\in C$). In this model, the demand can be from 1 to 4 Starships. This value is taken as an indicative amount for food orders to private households, solely for the purpose of this thesis.

- $\boldsymbol{B_o^a}$: matrix of values that indicate whether an order is available in arc a

$$\begin{cases} 1: If\ order\ o\ is\ available\ in\ \ arc\ a \\ 0: Otherwise \end{cases} \qquad \forall o \in Orders\ , \forall a\ \in Al$$

4.1.3 Decision Variables:

- $\boldsymbol{y_{ot}}$: is a variable that indicates whether an order is available in run t

$$\begin{cases} 1: \ If\ order\ o\ is\ available\ in\ run\ t \\ 0: \ Otherwise \end{cases} \qquad \forall o \in Orders, \forall t \in T$$

4.1.4 Objective function

Maximize the number of customers served as early as possible.

$$\textbf{Maximize:} \quad \sum_{o \in Orders} \sum_{t \in T} \left[\frac{(19*60)-t}{(19*60)} * y_{ot} \right]$$

Since the objective function aims to maximize the number of served customers, the model searches for the paths that achieve this. However, this approach may consider metro or train runs that are significantly distant from the actual order time, which is not feasible in real time due to the need for client satisfaction.

To address this issue, a new factor **((19∗60)−t)/(19∗60)** is introduced in the objective function. This factor measures the time lag between the current run (t) and the latest possible run (19 hours*60 minutes), relative to that latest run.

To maximize the objective function, the numerator **((19∗60)−t)∗y_{ot}** must be greater. When the numerator is higher, it indicates that the order has been assigned to a more immediate run and/or that y_{ot} is higher. This optimizes the model according to our goals.

In this way we obtain runs that are not necessarily too late and we can predict that the orders will arrive in reasonable timing.

4.1.5 Constraints

1. **Capacity Constraint for Each Metro/Tram Run:** The sum of orders in each arc in every metro/tram run must be less than or equal to the capacity of that arc.

$$\sum_{o \epsilon Orders} B_o^a * y_{ot} * d_o \leq cap_a \quad \forall a \in Al \ , \ \forall t \in T$$

2. **Ensure only one run is assigned to each order**:

$$\sum_{o \in Orders} y_{ot} \leq 1 \quad \forall t \in T$$

This model translates to a python script by using the Spyder interface in the Anaconda package (anaconda.com).

4.2 Python code and logic:

4.2.1 Generation of the schedule (T)

As described in the previous chapter, the timetable is categorized into peak and off-peak schedules. For the peak schedule the frequency is 10 minutes, while the off-peak frequency is 20 minutes. Accordingly, at the moment of the order, a set T is generated in python to include runs that come at the predefined frequencies (Figure 3).

```python
# Define peak and off-peak times
peak_start = time(7, 0)  # 7 AM
peak_end = time(19, 0)   # 7 PM
peak_frequency = 10       # 10 minutes during peak time
off_peak_frequency = 20  # 20 minutes during off-peak time

# Function to get the current time
def get_current_time():
    return datetime.now().time()

# Function to check if it's peak time
def is_peak_time(current_time):
    return peak_start <= current_time <= peak_end

# Function to generate the timetable based on peak/off-peak times
def generate_timetable():
    current_time = get_current_time()

    if is_peak_time(current_time):
        print("Peak time schedule activated.")
        frequency = peak_frequency
    else:
        print("Off-peak schedule activated.")
        frequency = off_peak_frequency

    return list(range(0, 19 * 60, frequency))

# Assign timetable based on the current time
T = generate_timetable()
```

Figure 3: schedule generation *(data, APPENDIX)*

4.2.2 Sample population

The sample population for this model is given by hypothetical restaurants and customers (Figure 4).

```python
# Function to generate points with a normal distribution
def generate_points(center, num_points, sigma):
    return [(np.random.normal(center[0], sigma),
             np.random.normal(center[1], sigma)) for _ in range(num_points)]

# Generate restaurants and customers around each station
restaurants = []
customers = []

sigma = 0.01  # Standard deviation for normal distribution
for mstation in metro_stations.values():
    restaurants.extend(generate_points(mstation, 5, sigma))
    customers.extend(generate_points(mstation, 10, sigma))
for tstation in tram_stations.values():
    restaurants.extend(generate_points(tstation, 5, sigma))
    customers.extend(generate_points(tstation, 10, sigma))
```

Figure 4: restaurants and customers generation *(data, APPENDIX)*

Furthermore, for simplicity, out of the generated restaurants and customers, only 10 randomly selected restaurants and 10 customers are used in the model as the sample population (Figure 5 & 6).

```
def reduceTo(lista,numb):
    newlist=[]
    if len(lista)>numb:
        inserted=0
        while inserted<numb:
            rand=random.randint(1,len(lista)-1)
            if not(lista[rand] in newlist):
                newlist.append(lista[rand])
                inserted=inserted+1

    else: return lista
    return newlist

restaurants=reduceTo(restaurants,10)
customers=reduceTo(customers,10)
```

Figure 5: sample population *(data, APPENDIX)*

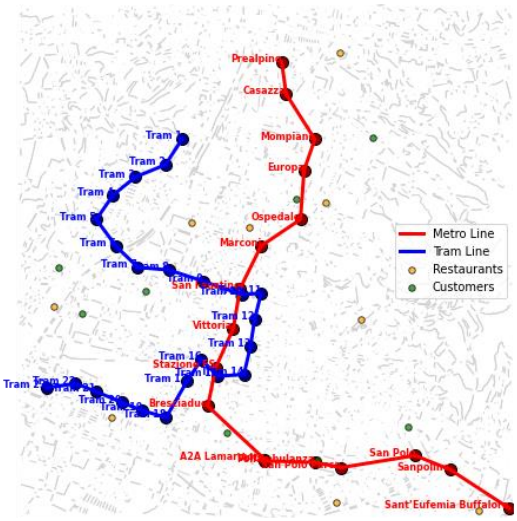

Figure 6: sample population *(data {output}, APPENDIX)*

4.2.3 Creation of Orders

Once we have the 'restaurants'(R) and the 'customers' (C) sets we can proceed to create orders. The orders are services from restaurants to customers. In the model they are taken as a set of pairs (r, c) where r is a restaurant and c is a customer. Only pairs that have a distance greater than 1.6 kms between them are included in the model (Figure 4). Starships can travel a maximum distance of 3,2 kms which would include a trip from the restaurant to the customer and back. That is why we consider 1.6 kms as a threshold.

```python
# Generate orders between restaurants and customers if the distance is > 1.6 km
Orders = [(i, j) for i in restaurants for j in customers if distance(i, j) > 1.6]

OrdersName=[]
k=0
orname='OrderN_'
for o in Orders:
    name=orname+str(k)
    k=k+1
    OrdersName.append(name)
def get_key_from_value(d, val):
    keys = [k for k, v in d.items() if v == val]
    if keys:
        return keys[0]
    return None
```

Figure 4: orders creation *(data, APPENDIX)*

4.2.4 Arcs

In the model, to better benefit of a path assignment algorithm, both metro line and tram lines are considered as one system of stations (nodes) and arcs, with their respective transfer stations (Figure 6).

Two key stations are identified as the transfer stations, since they fall on both lines. These stations are San Faustino station (Tram 10) and Stazione FS station (Tram 18).

The set *Al*, arranged in ascending order, includes:

- the metro_arcs, which extend from one station to the next along the metro line, in forward and reverse directions;
- the tram_arcs which extend from one station to the next along the tram line, in forward and reverse directions;
- the 'transfert_arcs' which consist in the transferring arcs between the metro line to the tram line and viceversa.

```python
# Define arcs for metro and tram lines
metro_arcs_back = [{'start': station1, 'end': station2, 'capacity': 50}
                for station1, station2 in zip(list(metro_stations.keys())[:-1], list(metro_stations.keys())[1:])]
# Define arcs for metro and tram lines
metro_arcs_forth = [{'start': station2, 'end': station1, 'capacity': 50}
                for station1, station2 in zip(list(metro_stations.keys())[:-1], list(metro_stations.keys())[1:])]


tram_arcs_back = [{'start': station1, 'end': station2, 'capacity': 50}
               for station1, station2 in zip(list(tram_stations.keys())[:-1], list(tram_stations.keys())[1:])]
tram_arcs_forth = [{'start': station2, 'end': station1, 'capacity': 50}
               for station1, station2 in zip(list(tram_stations.keys())[:-1], list(tram_stations.keys())[1:])]
transfert_arcs=[]
collegamento1={'start': 'San Faustino', 'end': 'Tram 10', 'capacity':3000}
collegamento2={'start': 'Tram 10', 'end': 'San Faustino', 'capacity':3000}
collegamento3={'start': 'Stazione FS', 'end': 'Tram 18', 'capacity':3000}
collegamento4={'start': 'Tram 18', 'end': 'Stazione FS', 'capacity':3000}
transfert_arcs.append(collegamento1)
transfert_arcs.append(collegamento2)
transfert_arcs.append(collegamento3)
transfert_arcs.append(collegamento4)

# Combine metro and tram arcs into a single dictionary
arcs = []

for arc in metro_arcs_back:

    arcs.append(arc)

for arc in tram_arcs_back:

    arcs.append(arc)
for arc in metro_arcs_forth:

    arcs.append(arc)

for arc in tram_arcs_forth:

    arcs.append(arc)
for arc in transfert_arcs:

    arcs.append(arc)
```

Figure 6: Arcs *(data, APPENDIX)*

Now that Arcs and Orders have been created, it is possible to identify possible paths for the orders.

4.2.5 Djikstra's algorithm & the B matrix

Djikstra's algorithm is a fundamental method used for finding the shortest path between nodes in a graph. It iteratively selects nodes with the smallest known distance, updating its neighbouring nodes with potential shorter paths. It then repeats this process untill all nodes have been evaluated (Dijkstra, E. W., 1959).

In the context of the food delivery optimization model presented in the thesis, Dijkstra's algorithm is used in determining the most efficient delivery routes for autonomous Starship robots. Specifically, the algorithm helps identify the shortest path for these robots as they navigate from restaurants to metro or tram stations, and then from those stations to customers'

locations. By using this algorithm paths for each order are created. Each path is then used to mark the arcs in which the path exists.

The Binary matrix has dimensions Arcs x Orders. As the python program assigns paths to the orders, in arcs where the order passes , it marks 1 in the B matrix. Otherwise the values are 0 (Figure 7).

```
# Generate correspondence with Metro arcs
B = np.zeros((len(archi), len(Orders)))

for o in range(len(Orders)):

    rest=Orders[o][0]
    cust=Orders[o][1]

    minLength=1000000000
    for mstation in metro_stations.values():

        length=distance(rest,mstation)
        if length<minLength:
            nodeStart=get_key_from_value(metro_stations, mstation)
            minLength=length

    for tstation in tram_stations.values():
        length=distance(rest,tstation)
        if length<minLength:
            nodeStart=get_key_from_value(tram_stations, tstation)
            minLength=length

    minLength=1000000000
    for mstation in metro_stations.values():

        length=distance(cust,mstation)
        if length<minLength:
            nodeEnd=get_key_from_value(metro_stations, mstation)
            minLength=length

    for tstation in tram_stations.values():
        length=distance(cust,tstation)
        if length<minLength:
            nodeEnd=get_key_from_value(tram_stations, tstation)
            minLength=length

    traversed_arcs = graph.dijkstra(nodeStart, nodeEnd)


    for a in range(len(traversed_arcs)):
        travarc=traversed_arcs[a]
        for arc in range(len(archi)):
            if (archi[arc]==travarc):
                B[arc][o]=1
```

Figure 7: Paths for the orders *(data, APPENDIX)*

4.2.6 Running the Model

We must then create the model in python as follows:

```
# Define the model
model = pyo.ConcreteModel()

# Decision variables
model.y = pyo.Var(Orders, T, domain=Binary)

model.obj= pyo.Objective(expr=sum((19*60-t)/(19*60)*model.y[o, t] for o in Orders for t in T), sense=maximize)

#Define Constraints

#capacity constraint
model.constraints = ConstraintList()
for t in T:
    for a in range(len(Al)):
        isT=False
        for o in range(len(Orders)):
            if B[a,o]>0:
                isT=True
        model.constraints.add(sum(B[a,o] * model.y[Orders[o], t]* d_p[o] for o in range(len(Orders))) <= 20)
        '''if isT==True:
            print(sum(B[a,o] * model.y[Orders[o], t] * d_p[o] for o in range(len(Orders))) <= 20)'''

#Run assignment constraint
for o in Orders:
    model.constraints.add(sum(model.y[o, t] for t in T) <= 1)
    #print(sum(model.y[o, t] for t in T) <= 1)

# Solve the model
SOLVER_NAME='glpk'

TIME_LIMIT=60

solver = SolverFactory(SOLVER_NAME)
```

Figure 8: Paths for the orders *(Model, APPENDIX)*

For convenience, a 60-second time limit is set for the model to run and produce the optimal results.

5. Analysis

The optimized delivery method has the following run assignments:

order OrderN_0 is assigned to run 20
order OrderN_1 is assigned to run **40**
order OrderN_2 is assigned to run 10
order OrderN_3 is assigned to run 40
order OrderN_4 is assigned to run 0
order OrderN_5 is assigned to run 10
order OrderN_6 is assigned to run 20
order OrderN_7 is assigned to run 0
order OrderN_8 is assigned to run 40
order OrderN_9 is assigned to run 10
order OrderN_10 is assigned to run 0
order OrderN_11 is assigned to run 0
order OrderN_12 is assigned to run 10
order OrderN_13 is assigned to run 0
order OrderN_14 is assigned to run 0
order OrderN_15 is assigned to run 0
order OrderN_16 is assigned to run 0
order OrderN_17 is assigned to run 20
order OrderN_18 is assigned to run 0
order OrderN_19 is assigned to run 10
order OrderN_20 is assigned to run 0
order OrderN_21 is assigned to run 10
order OrderN_22 is assigned to run 0
order OrderN_23 is assigned to run 0
order OrderN_24 is assigned to run 0
order OrderN_25 is assigned to run 0
order OrderN_26 is assigned to run 0
order OrderN_27 is assigned to run 0
order OrderN_28 is assigned to run 20
order OrderN_29 is assigned to run 0
order OrderN_30 is assigned to run 0
order OrderN_31 is assigned to run 30
order OrderN_32 is assigned to run 30
order OrderN_33 is assigned to run 20
order OrderN_34 is assigned to run 20
order OrderN_35 is assigned to run 0
order OrderN_36 is assigned to run 0
order OrderN_37 is assigned to run 0
order OrderN_38 is assigned to run 10
order OrderN_39 is assigned to run 30
order OrderN_40 is assigned to run 30
order OrderN_41 is assigned to run 30
order OrderN_42 is assigned to run 0
order OrderN_43 is assigned to run 10
order OrderN_44 is assigned to run 40
order OrderN_45 is assigned to run 0
order OrderN_46 is assigned to run 10
order OrderN_47 is assigned to run 0
order OrderN_48 is assigned to run 10
order OrderN_49 is assigned to run 0
order OrderN_50 is assigned to run 10
order OrderN_51 is assigned to run 0
order OrderN_52 is assigned to run 0
order OrderN_53 is assigned to run 20
order OrderN_54 is assigned to run 10
order OrderN_55 is assigned to run 0
order OrderN_56 is assigned to run 20
order OrderN_57 is assigned to run 10
order OrderN_58 is assigned to run 0
order OrderN_59 is assigned to run 0
order OrderN_60 is assigned to run 0
order OrderN_61 is assigned to run 0
order OrderN_62 is assigned to run 20
order OrderN_63 is assigned to run 0
order OrderN_64 is assigned to run 20
order OrderN_65 is assigned to run 0
order OrderN_66 is assigned to run 10
order OrderN_67 is assigned to run 40
order OrderN_68 is assigned to run 0
order OrderN_69 is assigned to run 0
order OrderN_70 is assigned to run 20
order OrderN_71 is assigned to run 10
order OrderN_72 is assigned to run 0
order OrderN_73 is assigned to run 0
order OrderN_74 is assigned to run 20
order OrderN_75 is assigned to run 0
order OrderN_76 is assigned to run 0
order OrderN_77 is assigned to run 0
order OrderN_78 is assigned to run 10
order OrderN_79 is assigned to run 10
order OrderN_80 is assigned to run 10
order OrderN_81 is assigned to run 0
order OrderN_82 is assigned to run 10
order OrderN_83 is assigned to run 10

order OrderN_84 is assigned to run 0
order OrderN_85 is assigned to run 0
order OrderN_86 is assigned to run 10
order OrderN_87 is assigned to run 0
order OrderN_88 is assigned to run

Table 3: Run assignments (Model {output}, APPENDIX)

Based on these results, the latest run at which an order is shipped occurs at the 40th minute. This simulation is conducted during peak hours, specifically within the 7 AM to 7 PM timeframe. According to the T timetable, runs are scheduled every 10 minutes. Therefore, the results show that the latest run utilized for shipment is the fourth run.

The model also provides insight into which orders populate each arc of every run. This is crucial for tracing back any problems that may occur along the shipping process.

Let's take the example of Order N_0 and see what the model suggests:

Arcs\Orders	**Order N_0**
...	0
5.Marconi →San Faustino	0
6.San Faustino→Vittoria	1
7.Vittoria→Stazione FS	1
8.StazFS→Bresciadue	1
9.Bresciadue→A2A Lamarmora	1
10.A2A Lamarmora→Volta	1
11.Volta→Poliambulanza	1
12.Poliambulanza→San Polo parco	1
13.San Polo parco→San Polo	1

14.San Polo→Sanpolino	1
15.Sanpolino→Sant'Eufemia Buffalora	1
16.Tram1→Tram2	0
17.Tram2→Tram3	0
18.Tram3→Tram4	0
19.Tram4→ Tram5	0
20.Tram5→Tram6	0
21.Tram6→Tram7	0
22.Tram7→Tram8	1
23.Tram8→Tram9	1
24.Tram9→Tram10	1
...	0
77.Tram 10→ San Faustino	1
78.StazioneFS→Tram 18	0
...	0

Table 4: B matrix extract (Model{output}, APPENDIX)

From the restaurant's location, Order N_0 reaches the closest station which is Tram 7(Figure 9). Here it onboards run 20 (Table 3), which is supposedly the second tram run. It rides along three tram arcs from Tram 7 to Tram 10 and gets transferred to the metro line, onboarding San faustino station. It then proceeds on the metro line reaching its final destination Sant'Eufemia Buffalora, from which it directly reaches the customer (Figure 9).

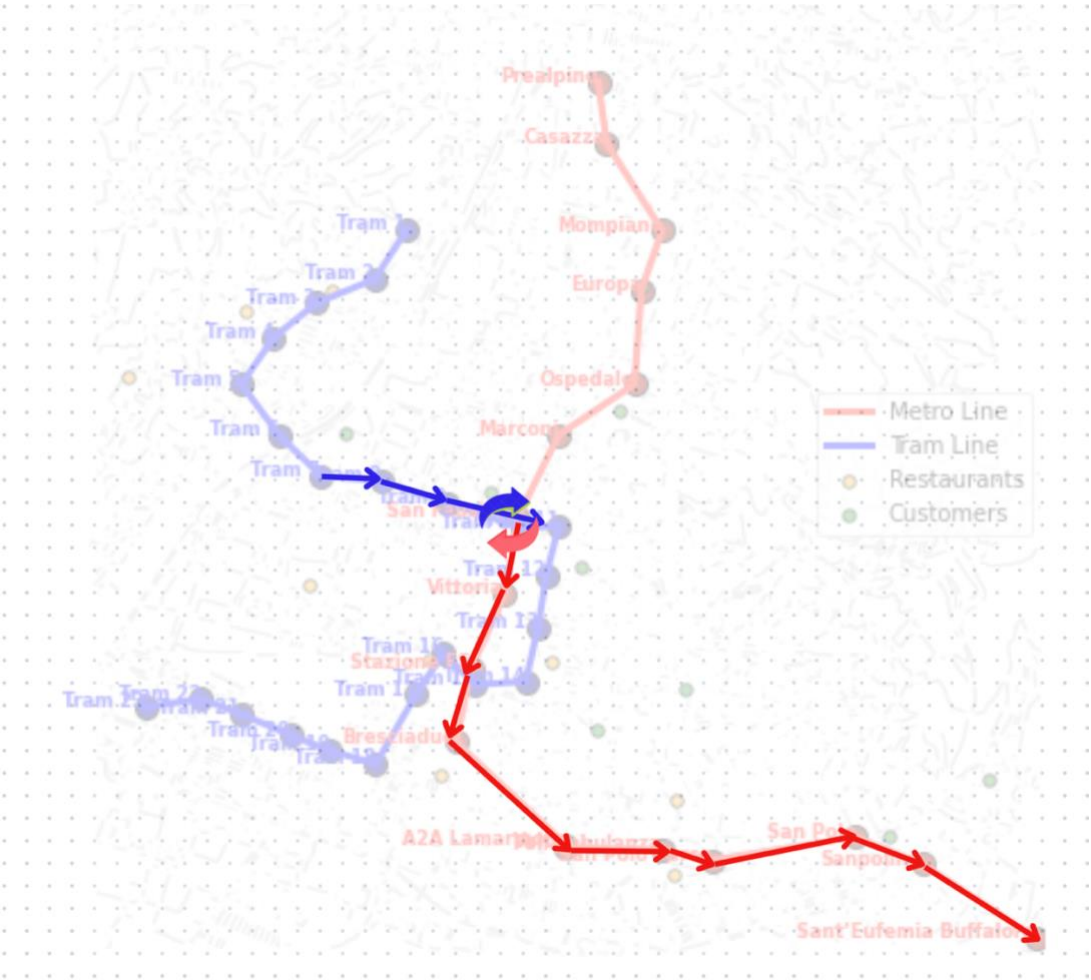

Figure 9: Order N_0 path

5.1 Peak vs. Off-peak schedules

The initial results obtained were calculated with 10 minute frequencies for the schedule.

The results obtained in table 5, show the run assignments in off-peak schedules where the runs have a frequency of 20 minutes:

order OrderN_0 is assigned to run 20	order OrderN_13 is assigned to run 40
order OrderN_1 is assigned to run 0	order OrderN_14 is assigned to run 0
order OrderN_2 is assigned to run 20	order OrderN_15 is assigned to run 20
order OrderN_3 is assigned to run 0	order OrderN_16 is assigned to run 0
order OrderN_4 is assigned to run 20	order OrderN_17 is assigned to run 0
order OrderN_5 is assigned to run 0	order OrderN_18 is assigned to run 0
order OrderN_6 is assigned to run 0	order OrderN_19 is assigned to run 0
order OrderN_7 is assigned to run 0	order OrderN_20 is assigned to run 0
order OrderN_8 is assigned to run 20	order OrderN_21 is assigned to run 0
order OrderN_9 is assigned to run 0	order OrderN_22 is assigned to run 0
order OrderN_10 is assigned to run 0	order OrderN_23 is assigned to run 20
order OrderN_11 is assigned to run 0	order OrderN_24 is assigned to run 0
order OrderN_12 is assigned to run 20	order OrderN_25 is assigned to run 0

order OrderN_26 is assigned to run 40	order OrderN_52 is assigned to run 0
order OrderN_27 is assigned to run 40	order OrderN_53 is assigned to run 0
order OrderN_28 is assigned to run 0	order OrderN_54 is assigned to run 0
order OrderN_29 is assigned to run 0	order OrderN_55 is assigned to run 0
order OrderN_30 is assigned to run 0	order OrderN_56 is assigned to run 0
order OrderN_31 is assigned to run 0	order OrderN_57 is assigned to run 0
order OrderN_32 is assigned to run 20	order OrderN_58 is assigned to run 0
order OrderN_33 is assigned to run 0	order OrderN_59 is assigned to run 20
order OrderN_34 is assigned to run 20	order OrderN_60 is assigned to run 0
order OrderN_35 is assigned to run 0	**order OrderN_61 is assigned to run 100**
order OrderN_36 is assigned to run 20	order OrderN_62 is assigned to run 0
order OrderN_37 is assigned to run 0	order OrderN_63 is assigned to run 20
order OrderN_38 is assigned to run 20	order OrderN_64 is assigned to run 0
order OrderN_39 is assigned to run 0	order OrderN_65 is assigned to run 20
order OrderN_40 is assigned to run 0	order OrderN_66 is assigned to run 0
order OrderN_41 is assigned to run 0	order OrderN_67 is assigned to run 20
order OrderN_42 is assigned to run 0	order OrderN_68 is assigned to run 0
order OrderN_43 is assigned to run 20	order OrderN_69 is assigned to run 60
order OrderN_44 is assigned to run 40	order OrderN_70 is assigned to run 20
order OrderN_45 is assigned to run 0	order OrderN_71 is assigned to run 0
order OrderN_46 is assigned to run 40	order OrderN_72 is assigned to run 40
order OrderN_47 is assigned to run 0	order OrderN_73 is assigned to run 0
order OrderN_48 is assigned to run 0	order OrderN_74 is assigned to run 0
order OrderN_49 is assigned to run 0	order OrderN_75 is assigned to run 40
order OrderN_50 is assigned to run 40	order OrderN_76 is assigned to run 0
order OrderN_51 is assigned to run 0	order OrderN_77 is assigned to run

Table 5: Run assignments (data {output}, APPENDIX)

In the off-peak schedule, the latest run assigned to an order occurs at the 100-minute mark, corresponding to the fifth run. This is more than double the maximum of 40 minutes observed during the peak schedule. This significant difference highlights the impact of metro/tram schedules on delivery efficiency, with off-peak operations resulting in noticeably longer service times compared to peak hours.

5.2 Capacity and Frequency Analysis

To further optimize the results of the model, the metro's capacity could be increased beyond 20. This adjustment would allow each run to accommodate more Starships, potentially speeding up the shipment process for all orders.

During the off-peak schedule, increasing the capacity to 25 allows the latest run to occur at the 60-minute mark. If the capacity is further raised to 30, the latest run can be completed at the 20-minute mark, which represents the optimal scenario achieved thus far.

Another potential avenue for optimization would be to increase the frequencies of metro and tram runs, enabling orders to be shipped earlier.

These combinations illustrate that for efficient delivery, both the capacity of the infrastructure and the frequency of runs significantly enhance the number of orders delivered.

5.3 Limitations of the model

Some of the limitations of this model are that the model assumes that the metro and tram schedules are syncronized and that there will be no delays, technical failures or disruptions in public transport operations. In real-world scenarios, these factors could affect delivery efficiency.

Furthermore, the model assumes that all infrastructure is accessible for Starship robots. In a more practical view, the moving stairs and the elevators at the stations may need additional features to accommodate Starships. Crowded pedestrian areas may also slow down the Starship's performance in delivery.

It is also suggested that the capacity may be increased from 20 upwards, but this idea assumes that there is space available, without considering how busy the runs are or the budget of Brescia mobilità.

It is therefore crucial to consider the applicability of such model in the discussed context.

Conclusion:

This thesis has analyzed the optimization of food delivery systems in Brescia by incorporating autonomous robots within the city's infrastructure. The primary objective of the model used in this study is to maximize the number of orders served while minimizing delivery time. By using a linear programming model, the study demonstrates that public infrastructure can immensely enhance delivery efficiency and provide an environmentally friendly alternative to traditional delivery methods.

The scheduling of public infrastructure, however, doesn't align well with the demand for food delivery services. People typically order food at night or in the late evenings when they're off work, meaning off-peak scheduling may not be suitable for accommodating delivery needs. As the results showed, during peak hours, with metro and tram runs scheduled every 10 minutes, most orders were served efficiently within 40 minutes, corresponding to the fourth run. In contrast, during off-peak hours, where the frequency of runs decreases to every 20 minutes, delivery times extended significantly, with the latest run occurring at the 100-minute mark. This variation between peak and off-peak schedules underscores the importance of proper scheduling in optimizing delivery times.

Additionally, the study highlights the potential benefits of increasing the metro's capacity beyond the current limit of 20 Starships per run. Accommodating more delivery robots on each run could further reduce delivery times and serve more customers, particularly during periods of high demand. But to be able to reach such conclusions a macro-level analysis for the costs of scaling should be done. Moreover, a study on the human experience of travelling alongside Starships should be conducted.

It is important to recognize that this simplified model has limitations in real-life applications. Future research should address these limitations and aim to develop a more comprehensive implementation framework for the model.

References:

Arrow, K. J. (1963). *Social choice and individual values*. Yale University Press.

Asdecker, B. (2021). Building the e-commerce supply chain of the future: What influences consumer acceptance of alternative places of delivery on the last-mile. *Logistics, 5*(4), 90. https://doi.org/10.3390/logistics5040090

Brescia Mobilità. https://www.bresciamobilita.it [Accessed 12 August 2024]

Comune Brescia. Approvazione progetto definitivo per la realizzazione della tramvia «T2 Pendolina-Fiera». https://www.comune.brescia.it/sites/default/files/2024-05/Tram%20Delibera%20progetto%20definitivo%20-%20Il%20progetto.pdf [Accessed 15 August 2024].

Debreu, G. (1959). *Theory of value: An axiomatic analysis of economic equilibrium*. Yale University Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271. https://doi.org/10.1007/BF01386390

Gevaers, R., Van de Voorde, E., & Vanelslander, T. (2011). Characteristics and challenges of last-mile logistics in an urban context. In *Proceedings of the 7th European Transport Conference*.

Holguín-Veras, J., Amaya-Leal, J., Cantillo, V., & Pérez-Bernabeu, E. (2018). Integrating freight into passenger transportation systems: An overview of city logistics initiatives. *Transportation Research Part A: Policy and Practice, 118*, 290-306. https://doi.org/10.1016/j.tra.2018.08.025

Kochan, B., Machemehl, R., & Zlatkovic, M. (2020). Integrating public transport and freight movement in urban areas. *Transportation Research Part A: Policy and Practice, 134*, 14-29. https://doi.org/10.1016/j.tra.2020.01.004

Li, W., Zhang, H., & Chen, L. (2019). Urban freight transport and public transit: New solutions for sustainable cities. *Journal of Urban Logistics, 8*(2), 97-108. https://doi.org/10.1016/j.jul.2019.05.011

MOOVIT. Orari e mappe della linea METROBS. https://moovitapp.com/index/en/public_transit-line-metrobs-Milano_e_Lombardia-223-3758536-178798267-0 [Accessed 1 July 2024].

Murray, C. C., & Chu, A. G. (2015). The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies, 54*, 86-109.

Pelletier, S., Jabali, O., & Laporte, G. (2016). Goods distribution with electric vehicles: Review and research perspectives. *European Journal of Operational Research, 247*(3), 707-720.

Pigou, A. C. (1920). *The economics of welfare*. Macmillan.

Rodríguez, J.-P., Comtois, C., & Slack, B. (2016). *The geography of transport systems* (4th ed.). Routledge.

Reyes, D., Savelsbergh, M., & Toriello, A. (2017). Vehicle routing with roaming delivery locations. *Transportation Research Part C: Emerging Technologies, 80*, 71-91.

Sánchez-Díaz, I., & van der Hurk, E. (2018). Freight-passenger co-modality in urban transport: Results from a pilot project. *Journal of Urban Logistics, 7*(1), 45-62. https://doi.org/10.1016/j.jul.2018.01.003

Starship Technologies. The Starship robot: Self-driving delivery robots. https://www.starship.xyz/the-starship-robot/ [Accessed 15 June 2024].

United Nations Environment Programme (UNEP). (2011). *Towards a green economy: Pathways to sustainable development and poverty eradication*. UNEP.

Varian, H. R. (1992). *Microeconomic analysis* (3rd ed.). W.W. Norton & Company.

Wang, Y., Zhang, J., Assogba, K., Liu, Y., Xu, M., & Wang, Y. (2018). Collaboration and transportation resource sharing in multiple centers vehicle routing optimization with delivery and pickup. *Knowledge-Based Systems, 160*, 296–310. https://doi.org/10.1016/j.knosys.2018.07.024

APPENDIX

data:-

```
from datetime import datetime, time
import heapq
import numpy as np
from numpy import random
import random
import osmnx as ox
import matplotlib.pyplot as plt
```

```
import numpy as np
from datetime import datetime, time
from geopy.distance import geodesic  # Correct import of the distance function


# Define the bounding box (Brescia area)
north, south, east, west = 45.59000, 45.51000, 10.28100, 10.17000
bbox = (north, south, east, west)

# Metro and Tram Stations (Same as your original code)
metro_stations = {
   'Prealpino': (45.58099, 10.22928),
   'Casazza': (45.57598, 10.23030),
   'Mompiano': (45.56894, 10.23669),
   'Europa': (45.56413, 10.23429),
   'Ospedale': (45.55646, 10.23349),
   'Marconi': (45.55231, 10.22468),
   'San Faustino': (45.54567, 10.22000),
   'Vittoria': (45.53942, 10.21847),
   'Stazione FS': (45.53334, 10.21455),
   'Bresciadue': (45.52736, 10.21280),
   'A2A Lamarmora': (45.51906, 10.22550),
   'Volta': (45.51882, 10.22542),
   'Poliambulanza': (45.51859, 10.23673),
   'San Polo Parco': (45.51778, 10.24255),
   'San Polo': (45.51966, 10.25910),
   'Sanpolino': (45.51746, 10.26689),
   'Sant'Eufemia Buffalora': (45.51145, 10.28007)
}

tram_stations = {
    'Tram 1': (45.569, 10.20707),
    'Tram 2': (45.56498, 10.20334),
    'Tram 3': (45.56305, 10.1966),
    'Tram 4': (45.56026, 10.19167),
    'Tram 5': (45.55645, 10.1879),
    'Tram 6': (45.55224, 10.19236),
    'Tram 7': (45.54899, 10.19717),
    'Tram 8': (45.54857, 10.20412),
    'Tram 9': (45.54683, 10.21176),
    'Tram 10': (45.54472, 10.22047),
    'Tram 11': (45.54497, 10.22472),
    'Tram 12': (45.54094, 10.22326),
    'Tram 13': (45.53667, 10.22232),
    'Tram 14': (45.53228, 10.22086),
    'Tram 15': (45.53203, 10.21506),
    'Tram 16': (45.53461, 10.21136),
    'Tram 17': (45.53126, 10.20817),
    'Tram 18': (45.52559, 10.20349),
    'Tram 19': (45.52667, 10.19817),
    'Tram 20': (45.52795, 10.19355),
    'Tram 21': (45.52965, 10.1878),
    'Tram 22': (45.53088, 10.18325),
    'Tram 23': (45.53021, 10.17677)
}

# Define peak and off-peak times
peak_start = time(7, 0)  # 7 AM
peak_end = time(19, 0)   # 7 PM
peak_frequency = 10       # 10 minutes during peak time
off_peak_frequency = 20  # 20 minutes during off-peak time

# Function to get the current time
def get_current_time():
    return datetime.now().time()

# Function to check if it's peak time
def is_peak_time(current_time):
    return peak_start <= current_time <= peak_end

# Function to generate the timetable based on peak/off-peak times
def generate_timetable():
    current_time = get_current_time()

    if is_peak_time(current_time):
```

```
        print("Peak time schedule activated.")
        frequency = peak_frequency
    else:
        print("Off-peak schedule activated.")
        frequency = off_peak_frequency

    return list(range(0, 19 * 60, frequency))

# Assign timetable based on the current time
T = generate_timetable()

# Define arcs for metro and tram lines
metro_arcs_back = [{'start': station1, 'end': station2, 'capacity': 50}
            for station1, station2 in zip(list(metro_stations.keys())[:-1], list(metro_stations.keys())[1:])]
# Define arcs for metro and tram lines
metro_arcs_forth = [{'start': station2, 'end': station1, 'capacity': 50}
            for station1, station2 in zip(list(metro_stations.keys())[:-1], list(metro_stations.keys())[1:])]


tram_arcs_back = [{'start': station1, 'end': station2, 'capacity': 50}
            for station1, station2 in zip(list(tram_stations.keys())[:-1], list(tram_stations.keys())[1:])]
tram_arcs_forth = [{'start': station2, 'end': station1, 'capacity': 50}
            for station1, station2 in zip(list(tram_stations.keys())[:-1], list(tram_stations.keys())[1:])]
transfert_arcs=[]
collegamento1={'start': 'San Faustino', 'end': 'Tram 10', 'capacity':3000}
collegamento2={'start': 'Tram 10', 'end': 'San Faustino', 'capacity':3000}
collegamento3={'start': 'Stazione FS', 'end': 'Tram 15', 'capacity':3000}
collegamento4={'start': 'Tram 15', 'end': 'Stazione FS', 'capacity':3000}
transfert_arcs.append(collegamento1)
transfert_arcs.append(collegamento2)
transfert_arcs.append(collegamento3)
transfert_arcs.append(collegamento4)

# Combine metro and tram arcs into a single dictionary
arcs = []

for arc in metro_arcs_back:

    arcs.append(arc)

for arc in tram_arcs_back:

    arcs.append(arc)
for arc in metro_arcs_forth:

    arcs.append(arc)

for arc in tram_arcs_forth:

    arcs.append(arc)
for arc in transfert_arcs:

    arcs.append(arc)

# Download the street network within the bounding box
G = ox.graph_from_bbox(north, south, east, west, network_type='all')

# Plot the street network with metro and tram lines
fig, ax = ox.plot_graph(G, bbox=bbox, node_color="white", edge_color="lightgray", bgcolor="white", show=False, close=False)

metro_x, metro_y = zip(*metro_stations.values())
ax.plot(metro_y, metro_x, color='red', linewidth=3, label='Metro Line')
ax.scatter(metro_y, metro_x, color='darkred', s=100, marker='o', edgecolor='black')

tram_x, tram_y = zip(*tram_stations.values())
ax.plot(tram_y, tram_x, color='blue', linewidth=3, label='Tram Line')
ax.scatter(tram_y, tram_x, color='darkblue', s=100, marker='o', edgecolor='black')

# Function to generate points with a normal distribution
def generate_points(center, num_points, sigma):
    return [(np.random.normal(center[0], sigma),
            np.random.normal(center[1], sigma)) for _ in range(num_points)]

# Generate restaurants and customers around each station
restaurants = []
```

```
customers = []

sigma = 0.01  # Standard deviation for normal distribution
for mstation in metro_stations.values():
    restaurants.extend(generate_points(mstation, 5, sigma))
    customers.extend(generate_points(mstation, 10, sigma))
for tstation in tram_stations.values():
    restaurants.extend(generate_points(tstation, 5, sigma))
    customers.extend(generate_points(tstation, 10, sigma))

def reduceTo(lista,numb):
    newlist=[]
    if len(lista)>numb:
        inserted=0
        while inserted<numb:
            rand=random.randint(1,len(lista)-1)
            if not(lista[rand] in newlist):
                newlist.append(lista[rand])
                inserted=inserted+1

    else: return lista
    return newlist

restaurants=reduceTo(restaurants,10)
customers=reduceTo(customers,10)



# Define ranges for restaurants, customers, lines, arcs
R = range(len(restaurants))
C = range(len(customers))
L = range(2)  # Metro & tram lines
Al = range(len(arcs))




class Graph:
    def __init__(self, arcs):
        """
        arcs: List of tuples (origin, destination, weight)
        """
        self.graph = {}
        for origin, destination, weight in arcs:
            if origin not in self.graph:
                self.graph[origin] = []
            if destination not in self.graph:
                self.graph[destination] = []
            self.graph[origin].append((destination, weight))

    def dijkstra(self, start, end):
        # Min-heap priority queue
        priority_queue = [(0, start, [])]  # (cumulative distance, node, path traversed)
        visited = set()
        shortest_paths = {start: (0, [])}  # {node: (cumulative distance, path)}

        while priority_queue:
            (current_distance, current_node, path) = heapq.heappop(priority_queue)

            if current_node in visited:
                continue
            visited.add(current_node)

            # If we reach the destination, return the path
            if current_node == end:
                return path

            # Traverse neighbors
            for neighbor, weight in self.graph.get(current_node, []):
                distance = current_distance + weight
                if neighbor not in shortest_paths or distance < shortest_paths[neighbor][0]:
                    shortest_paths[neighbor] = (distance, path + [(current_node, neighbor, weight)])
                    heapq.heappush(priority_queue, (distance, neighbor, path + [(current_node, neighbor, weight)]))
```

```
        return None  # If no path exists


# Function to calculate geodesic distance
def distance(point1, point2):
    return geodesic(point1, point2).km  # Returns distance in kilometers
# Example usage

archi=[]
for al in arcs:
    #print(al)
    nodeSt=al['start']
    #print(nodeSt)
    nodeFi=al['end']
    #print(nodeFi)
    dist=1
    #print(dist)
    arc=(al['start'],al['end'],1)
    archi.append(arc)


graph = Graph(archi)




# Generate orders between restaurants and customers if the distance is > 1.6 km
Orders = [(i, j) for i in restaurants for j in customers if distance(i, j) > 1.6]

OrdersName=[]
k=0
orname='OrderN_'
for o in Orders:
    name=orname+str(k)
    k=k+1
    OrdersName.append(name)
def get_key_from_value(d, val):
    keys = [k for k, v in d.items() if v == val]
    if keys:
        return keys[0]
    return None

# Generate correspondence with Metro arcs
B = np.zeros((len(archi), len(Orders)))

for o in range(len(Orders)):

    rest=Orders[o][0]
    cust=Orders[o][1]

    minLength=1000000000
    for mstation in metro_stations.values():

        length=distance(rest,mstation)
        if length<minLength:
            nodeStart=get_key_from_value(metro_stations, mstation)
            minLength=length

    for tstation in tram_stations.values():
        length=distance(rest,tstation)
        if length<minLength:
            nodeStart=get_key_from_value(tram_stations, tstation)
            minLength=length

    minLength=1000000000
    for mstation in metro_stations.values():

        length=distance(cust,mstation)
        if length<minLength:
            nodeEnd=get_key_from_value(metro_stations, mstation)
            minLength=length

    for tstation in tram_stations.values():
        length=distance(cust,tstation)
```

```
        if length<minLength:
            nodeEnd=get_key_from_value(tram_stations, tstation)
            minLength=length

    traversed_arcs = graph.dijkstra(nodeStart, nodeEnd)


    for a in range(len(traversed_arcs)):
        travarc=traversed_arcs[a]
        for arc in range(len(archi)):
            if (archi[arc]==travarc):
                B[arc][o]=1 
```

```
# Generate demand for each path (restaurant to customer pair)
d_p=[]
for o in range(len(Orders)):
    nn = np.random.randint(1, 5)
    d_p.append(nn)
#print('Ecco d_p')
#print(d_p)
# Get data function to return necessary information
def get_data():
    return {
        'R': restaurants,
        'C': customers,
        'Orders': OrdersName,
        'T': T,
        'L': L,
        'Al': arcs,
        'd_p': d_p,
        'B': B,
    }

# Plot restaurants and customers
restaurants_x, restaurants_y = zip(*restaurants)
ax.scatter(restaurants_y, restaurants_x, color='orange', s=30, label='Restaurants', edgecolor='black', alpha=0.7)

customers_x, customers_y = zip(*customers)
ax.scatter(customers_y, customers_x, color='green', s=30, label='Customers', edgecolor='black', alpha=0.7)

# Add station labels
for station, (x, y) in metro_stations.items():
    ax.text(y, x, station, fontsize=8, color='red', ha='right', weight='bold')

for station, (x, y) in tram_stations.items():
    ax.text(y, x, station, fontsize=8, color='blue', ha='right', weight='bold')

# Customize plot
plt.legend()
plt.show()
```

Model:-

```
import pyomo.environ as pyo
from pyomo.environ import *
from pyomo.environ import ConstraintList
from pyomo.opt import SolverFactory
import data4 as dg  # Ensure this imports the correct data module


# Get generated data
```

```
data = dg.get_data()
#print("I have data")

# Extract data from the data dictionary
R = data['R']
C = data['C']
d_p = data['d_p']
#print(d_p)
Orders = data['Orders']
T = data['T']
B = data['B']
Al = data['Al']
#B=data['B']


# Define the model
model = pyo.ConcreteModel()

# Decision variables
model.y = pyo.Var(Orders, T, domain=Binary)

model.obj= pyo.Objective(expr=sum((19*60-t)/(19*60)*model.y[o, t] for o in Orders for t in T), sense=maximize)

#Define Constraints

#capacity constraint
model.constraints = ConstraintList()
for t in T:
    for a in range(len(Al)):
        isT=False
        for o in range(len(Orders)):
            if B[a,o]>0:
                isT=True
        model.constraints.add(sum(B[a,o] * model.y[Orders[o], t]* d_p[o] for o in range(len(Orders))) <= 20)
        '''if isT==True:
            print(sum(B[a,o] * model.y[Orders[o], t] * d_p[o] for o in range(len(Orders))) <= 20)'''

#Run assignment constraint
for o in Orders:
    model.constraints.add(sum(model.y[o, t] for t in T) <= 1)
    #print(sum(model.y[o, t] for t in T) <= 1)

# Solve the model
SOLVER_NAME='glpk'

TIME_LIMIT=60

solver = SolverFactory(SOLVER_NAME)


solver.options['mipgap'] = 0.01
if 'cplex' in SOLVER_NAME:
    solver.options['timelimit'] = TIME_LIMIT
elif 'glpk' in SOLVER_NAME:
    solver.options['tmlim'] = TIME_LIMIT
elif 'gurobi' in SOLVER_NAME:
    solver.options['TimeLimit'] = TIME_LIMIT
elif 'xpress' in SOLVER_NAME:
    solver.options['soltimelimit'] = TIME_LIMIT


solver.solve(model, tee=True)

# Print the results
for o in Orders:
    served=False
    for t in T:
        if pyo.value(model.y[o, t]) == 1:
            served=True
            print("order ",o, "is assigned to run ",t)
    if served==False:
        print("order ",o, "is not assigned to any run ")
for t in T:
    for a in range(len(Al)):
        occ=0
```

```
for o in range(len(Orders)):
    occ=occ+B[a,o] * pyo.value(model.y[Orders[o], t])* d_p[o]
if occ>0:
    print("Arc ",Al[a]," in run ",t, "usage ",str(occ),"/",str(20))
```